

Reading + Reproduction Report: Submodular Order Functions and Assortment Optimization

Reproduction Report

2026-02-28

Contents

1	Paper Overview	3
2	Core Definitions and Notation	3
2.1	(Strong) submodular order	3
2.2	Weak submodular order (used for the main approximation theorems)	3
3	Main Theoretical Results (Correct Theorem Numbering)	4
3.1	Cardinality constraint: $ S \leq k$ (Theorem 2)	4
3.2	Nearly tight hardness (Theorem 3)	4
3.3	Budget / knapsack constraint: $b(S) \leq B$ (Theorem 4)	4
3.4	Matroid constraint: $S \in \mathcal{I}$ (Theorem 5)	4
3.5	Noisy value oracles (robustness)	4
4	Algorithm Map (Correct Algorithm Numbering)	5
5	Applications in the Paper (Assortment + Streaming)	5
5.1	Assortment optimization via a monotone objective	5
5.2	Connection to streaming submodular maximization	5
6	What This Codebase Implements	6
7	Comparison: Paper Section 5 vs My Results	6
7.1	Experimental setup alignment	6
7.2	Qualitative agreement with paper claims	7

7.3 Per-k comparison table from My run	7
8 Final Assessment	8
9 Appendix: Full Detailed Summary	9
10 Reference	9

1 Paper Overview

The paper defines **submodular order functions**: monotone, subadditive set functions that satisfy a *directional* diminishing-returns property along a permutation (order) of the ground set. This structure is strictly weaker than full submodularity but strong enough to enable **constant-factor approximation algorithms** under common constraints (cardinality, budget/knapsack, matroid), and it yields new guarantees for **constrained assortment optimization** (MNL, MMNL-with-customization, and Markov/compatible models) as well as a clean connection to **streaming submodular maximization**.

2 Core Definitions and Notation

Let N be the ground set. For a permutation (order) π over N , index elements from left to right by π .

- For a set $S \subseteq N$, let $l_\pi(S)$ be the **leftmost** element of S in order π , and $r_\pi(S)$ be the **rightmost** element.
- For sets $X, S \subseteq N$, define the marginal value

$$f(X \mid S) := f(X \cup S) - f(S).$$

2.1 (Strong) submodular order

A permutation π is a **submodular order** for f if for all sets $B \subseteq A$ and for all sets C entirely to the right of A (i.e., $l_\pi(C) > r_\pi(A)$), we have

$$f(C \mid A) \leq f(C \mid B).$$

A function that admits at least one such π is called a **submodular order function**.

2.2 Weak submodular order (used for the main approximation theorems)

The approximation guarantees in the paper are proved for an even milder notion, **weak submodular order**.

- Sets $B \subseteq A$ are called **π -nested** if the leftmost element of $A \setminus B$ is to the right of B , i.e.

$$l_\pi(A \setminus B) > r_\pi(B).$$

- An order π is a **weak submodular order** if

$$f(C \mid A) \leq f(C \mid B)$$

for all π -nested $B \subseteq A$ and all sets C with $l_\pi(C) > r_\pi(A)$.

Intuitively: only “nested” pairs $B \subseteq A$ must satisfy the diminishing-returns condition; non-nested pairs impose no constraint.

3 Main Theoretical Results (Correct Theorem Numbering)

All results below assume f is **monotone** and **subadditive**, and the relevant (weak) submodular order is known.

3.1 Cardinality constraint: $|S| \leq k$ (Theorem 2)

Theorem 2. For cardinality-constrained maximization with a known weak submodular order, **Algorithm 1** achieves a

$$(1 - \varepsilon) \cdot 0.500$$

approximation using $O(\frac{n}{\varepsilon} \log k)$ value-oracle queries.

3.2 Nearly tight hardness (Theorem 3)

Theorem 3 (Upper bound). For (strong) submodular order functions, **no** algorithm making only $\text{poly}(n)$ value-oracle queries can guarantee better than

$$0.500 + \varepsilon$$

for the cardinality-constrained problem. In this oracle model, the 0.500-level guarantee is essentially best-possible.

3.3 Budget / knapsack constraint: $b(S) \leq B$ (Theorem 4)

Theorem 4. Under a budget constraint and a known weak submodular order: - **Algorithm 3** gives a fast $\frac{1-\varepsilon}{3}$ approximation (implemented via a bang-per-buck threshold scan). - **Algorithm 5** achieves a stronger $0.500 - \varepsilon$ approximation (it may *discard* previously added items via the Final Add subroutine).

3.4 Matroid constraint: $S \in \mathcal{I}$ (Theorem 5)

Theorem 5. For matroid-constrained maximization with known (weak) submodular order, **Algorithm 7** achieves a 0.250 approximation with query complexity $O(nd)$, where d is the matroid rank.

3.5 Noisy value oracles (robustness)

If the value oracle is multiplicatively noisy, i.e. $(1 - \delta)f(S) \leq \hat{f}(S) \leq (1 + \delta)f(S)$ for all S , the paper shows the above approximation guarantees degrade by an additional multiplicative factor of the form

$$1 - O\left(\frac{n\delta}{1 - \delta}\right),$$

so guarantees remain meaningful when δ is sufficiently small.

4 Algorithm Map (Correct Algorithm Numbering)

Below is a concise “what is where” map consistent with the paper’s numbering.

- **Algorithm 1:** Cardinality, threshold-grid wrapper (outer loop over thresholds).
- **Algorithm 2 (Threshold Add):** Single pass over N in order π , adding elements with marginal gain above a threshold.
- **Algorithm 3:** Budget $\left(\frac{1-\varepsilon}{3}\right)$ via *bang-per-buck* thresholding.
- **Algorithm 4 (B-Th Add):** The threshold-add subroutine used by Algorithm 3 under budget feasibility.
- **Algorithm 5:** Budget $(0.500 - \varepsilon)$ algorithm using the **Final Add** routine to handle “large” budget elements; can remove previously selected items.
- **Algorithm 6 (Final Add):** Feasibility/repair-style subroutine used inside Algorithm 5.
- **Algorithm 7:** Matroid (0.250) algorithm based on directed swap-chains.
- **Algorithm 8:** Framework for **compatible choice models**; constructs a relaxed order and then applies the above constrained-maximization algorithms.
- **Algorithm 9:** A modified version of the matroid algorithm (for strong submodular order) that uses marginals $f(j \mid S)$, enabling a streaming implementation.

5 Applications in the Paper (Assortment + Streaming)

5.1 Assortment optimization via a monotone objective

Because revenue $R_\phi(S)$ can be nonmonotone, the paper optimizes the monotone transform

$$f_\phi(S) = \max_{X \subseteq S} R_\phi(X).$$

Under **substitutability** (adding an unavailable item cannot increase the probability of choosing another available item), f_ϕ is monotone and subadditive, and a value oracle can be implemented by solving an unconstrained assortment problem on the reduced ground set S .

- **MNL:** Sorting by **descending price** is a (strong) submodular order for f_ϕ , so all constrained guarantees apply directly.
- **MMNL with customization:** f_ϕ decomposes as a nonnegative weighted sum across customer types; closure under addition yields the first constant-factor guarantees for general m .
- **Markov / compatible choice models:** Descending price need not be a submodular order, but Markov models are shown **compatible**, and **Algorithm 8** transfers the same guarantees (Theorem 7 and Table 2 in the paper).

5.2 Connection to streaming submodular maximization

Many of the paper’s algorithms parse elements **once** in a designated order. For a *submodular* function, every order is a submodular order, so these one-pass algorithms immediately imply streaming guarantees (summarized in Theorem 8 in the paper).

6 What This Codebase Implements

This repository reproduces the paper’s MNL + cardinality benchmark pipeline end-to-end, with a strong focus on **Algorithm 1 vs exact OPT**.

- `src/mnl.py`
- Generates the paper Section 5 style MNL instances: $r_i, \mu_i \sim U[15, 25]$, $v_i = \exp(\mu_i - r_i)$, $v_0 = 1$.
- Implements `order_by_price_desc`, matching the paper’s MNL order structure.
- Implements `MNLAppendOracle` for $O(1)$ incremental marginal checks.
- `src/alg1_cardinality.py`
- `threshold_add_mnl`: Algorithm 2 style threshold-add subroutine.
- `alg1_cardinality_mnl`: Algorithm 1 style threshold-grid wrapper.
- `src/opt_mnl_cardinality.py`
- Exact OPT solver via Dinkelbach iterations for cardinality-MNL objective.
- Used as numerical upper bound / ground-truth comparator.
- `experiments/*` and `results/stage5_end2end_small_trial5/summary.csv`
- Implements and records the ALG-vs-OPT benchmark in reproducible CSV format.
- `tests/*`
- Oracle correctness, ALG correctness/determinism, OPT correctness (vs brute force on small instances), and seed reproducibility.

For more details, please refer to the README.md in the GitHub repository.

7 Comparison: Paper Section 5 vs My Results

7.1 Experimental setup alignment

Item	Paper Section 5	My run
Problem	Cardinality-constrained MNL assortment; compare ALG vs OPT	Same
n	50,000	50,000
Price distribution r_i	Uniform[15, 25]	Uniform[15, 25]
Mean utility μ_i	Uniform[15, 25]	Uniform[15, 25]
Attraction v_i	$\exp(\mu_i - r_i)$	$\exp(\mu_i - r_i)$
Outside option v_0	1	1

Item	Paper Section 5	My run
k set	{1,10,80,600,2000,10000,25000,45000}	Same
Trials per k	1000	200
epsilon	0.100	0.100
Main metric	100*Average(ALG)/Average(OPT) vs log(k)	Same

The setup is highly aligned. The main difference is trials per k (paper: 1000, My run: 200), to save time.

7.2 Qualitative agreement with paper claims

From the paper's Section 5 text of the paper and Figure-1 notes:

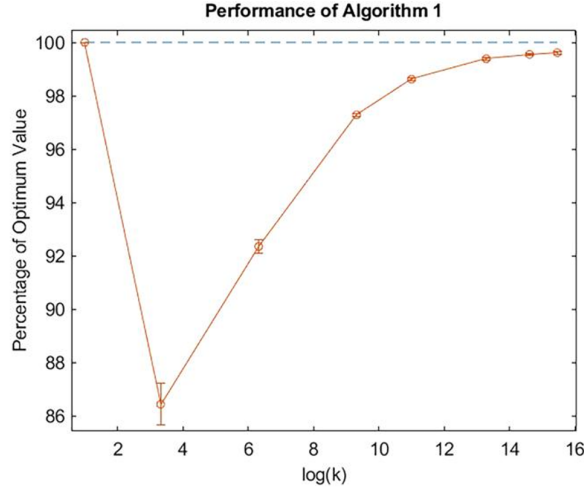
- ALG is very close to OPT when $k = 1$ and when k is close to n .
- Plot uses $100 \times \text{Average}(\text{ALG})/\text{Average}(\text{OPT})$ against $\log(k)$.
- Error bars become very small for larger k .
- Ratio-of-averages and average-of-ratios show no qualitative difference.

My summary matches all of these trends:

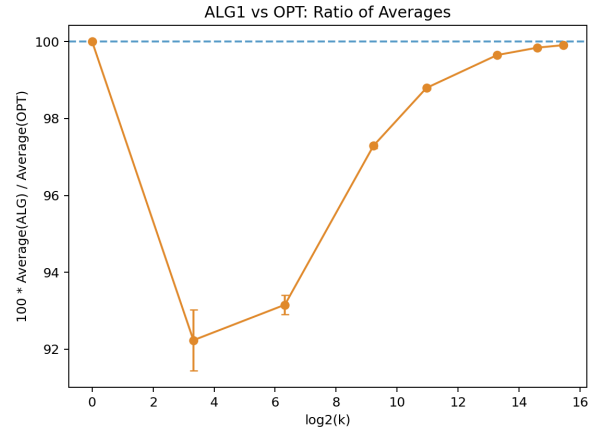
- At $k = 1$, `ratio_of_averages` = 1.000.
- At $k = 45000 \approx n$, `ratio_of_averages` = 0.999.
- Ratio increases toward 1 as k grows.
- `std_ratio` decreases from 0.008 ($k = 10$) to 0.000 ($k = 45000$).
- `ratio_of_averages` and `mean_ratio` are almost identical at every k .

7.3 Per-k comparison table from My run

k	ratio_of_averages	std_ratio	mean_ALG_ms	mean_OPT_ms
1	1.000	0.000	13.030	0.482
10	0.922	0.008	1148.961	3.151
80	0.932	0.003	2136.547	3.720
600	0.973	0.001	3043.056	5.104
2000	0.988	0.000	3594.666	7.407
10000	0.997	0.000	4328.267	13.182
25000	0.998	0.000	4840.972	16.519
45000	0.999	0.000	5076.503	15.431



(a) Original figure from the paper.



(b) My plot of ratio_of_averages vs. $\log(k)$.

Figure 1:

Note: the paper provides Figure-1 level trends and plotting notes, not a full numeric table for each k , so exact point-by-point residual comparison is not available from the published text alone.

8 Final Assessment

- The codebase faithfully reproduces the main Section-5 numerical benchmark setting for MNL cardinality-constrained assortment optimization.
- The observed ALG/OPT behavior is consistent with the paper’s experimental narrative.
- The implementation also goes beyond the paper’s qualitative plot by providing exact per- k CSV summaries and runtime fields.

9 Appendix: Full Detailed Summary

Table 9A. Objective quality summary

k	mean_ALG	std_ALG	mean_OPT	std_OPT	ratio_of_averages	std_ratio
1	20.948	0.006	20.948	0.006	1.000	0.000
10	21.335	0.184	23.131	0.007	0.922	0.008
80	22.832	0.062	24.509	0.014	0.932	0.003
600	24.030	0.013	24.699	0.010	0.973	0.001
2000	24.403	0.010	24.701	0.009	0.988	0.000
10000	24.616	0.010	24.702	0.009	0.997	0.000
25000	24.663	0.009	24.701	0.009	0.998	0.000
45000	24.677	0.010	24.700	0.010	0.999	0.000

Table 9B. Runtime / feasibility summary

k	pass_rate_ALG_le_OPT	mean_ALG_ms	mean_OPT_ms	mean_OPT_iters	n
1	1.000	13.030	0.482	1.000	200
10	1.000	1148.961	3.151	6.210	200
80	1.000	2136.547	3.720	7.335	200
600	1.000	3043.056	5.104	9.090	200
2000	1.000	3594.666	7.407	10.130	200
10000	1.000	4328.267	13.182	10.730	200
25000	1.000	4840.972	16.519	10.645	200
45000	1.000	5076.503	15.431	10.705	200

10 Reference

Rajan Udwania. *Submodular Order Functions and Assortment Optimization*. **Management Science** 71(1):202–218 (January 2025). Published online April 1, 2024. DOI: 10.129/mnsc.2021.041.